

21.动态shape输入

1. 动态Shape功能介绍

动态shape是指模型输入数据的形状在运行时可以改变。它可以帮助处理输入数据大小不固定的情况，增加模型的灵活性。在之前仅支持静态shape的RKNN模型情况下，如果用户需要使用多个输入shape，传统

的做法是生成多个RKNN模型，在模型部署时初始化多个上下文分别执行推理，而在引入动态shape后，

用户可以只保留一份与静态shape RKNN模型大小接近的动态shape RKNN模型，并使用一个上下文进行推理，从而节省Flash占用和DDR占用，动态shape在图像处理和序列模型推理中具有重要的作用。

应用场景包括：

- 序列长度改变的模型，常见于NLP模型，例如BERT, GPT。
- 空间维度变化的模型，例如分割和风格迁移。
- 带Batch模型，Batch维度上变化。
- 可变输出数量的目标检测模型。

2. 生成动态Shape的RKNN模型

使用RKNN-Toolkit2的Python接口生成动态shape的RKNN模型的步骤：

1.确认模型支持动态shape

如果模型文件本身不是动态shape, RKNN-Toolkit2支持扩展成动态shape的RKNN模型。首先，用户要确认模型本身不存在限制动态shape的算子或子图结构，例如，常量的形状无法改变，RKNN-Toolkit2工具在转换过程会报错，如果遇到不支持动态shape扩展的情况，用户要根据报错信息，修改模型结构，重新训练模型以支持动态shape。建议使用原始模型本身就是动态shape的模型。

2.设置需要使用的输入形状

由于NPU硬件特性，动态shape RKNN模型不支持输入形状任意改变，要求用户设置有限个输入形状。对

于多输入的模型，每个输入的shape个数要相同。

需要在rknn.config接口中开启**dynamic_input=dynamic_shapes**。例如，在使用RKNN-Toolkit2转换Caffe模型时，Python

代码示例如下：

```

1  # Python
2  dynamic_input = [
3      [[1,3,224,224]], # set the first shape for all inputs
4      [[1,3,192,192]], # set the second shape for all inputs
5      [[1,3,160,160]], # set the third shape for all inputs
6  ]
7
8  # Pre-process config
9  rknn.config(mean_values=[103.94, 116.78, 123.68],
10              std_values=[58.82, 58.82, 58.82],
11              quant_img_RGB2BGR=True,
12              dynamic_input=dynamic_shapes)

```

上述接口配置会生成支持3个shape分别是[1,3,224,224]、[1,3,192,192]和[1,3,160,160]的动态shape RKNN模型。

3.量化

在设置好输入shape后，如果要做量化，则需要设置量化矫正集数据。工具会读取用户设置的最大分辨率输入做量化（是所有输入尺寸之和的最大的一组shape）。例如，模型有两个输入，一个输入shape分别是[1,224]和[1,112]，另一个输入shape分别[1,40]和[1,80]，第一组shape所有输入尺寸之和是 $1*224+1*40=264$ ，第二组shape所有输入尺寸之和是 $1*112+1*80=192$ ，第一组shape所有输入尺寸之和更大，因此使用两个输入分别以[1,224]和[1,40]的shape做量化。

- 如果量化矫正集是jpg/png图片格式，用户可以使用不同的分辨率的图片做量化，因为工具会对图片

使用opencv的resize方法缩放到最大分辨率后做量化。

- 如果量化矫正集是numpy格式，则用户必须使用最大分辨率输入的shape。量化后，模型内所有shape在

运行时使用**同一套量化参数**进行推理。

另外，输入的最大分辨率shape在调用rknn.config时也会打印出来，如下：

代码块

```

1  W config: The 'dynamic_input' function has been enabled, the MaxShape is
    dynamic_input[0] = [[1,224],[1,40]]!
2  The following functions are subject to the MaxShape:
3  1. The quantified dataset needs to be configured according to MaxShape
4  2. The eval_perf or eval_memory return the results of MaxShape

```

4.推理评估或精度分析

动态shape RKNN模型做推理或做精度分析时，用户必须提供第2步中设置的其中一组shape的输入。接口

使用上与静态shape RKNN模型场景一致，此处不做赘述。

3. 代码测试

12_dynamic_shape

代码块

```
1  import numpy as np
2  import cv2
3  from rknn.api import RKNN # 导入RKNN API, 用于模型转换和推理
4
5  # 定义展示输出结果的函数
6  def show_outputs(outputs):
7      output_ = outputs[0].reshape((-1, 1000))
8      fp = open('./labels.txt', 'r')
9      labels = fp.readlines()
10     # 遍历每个输出 (处理批量推理的情况)
11     for output in output_:
12         # 对输出值进行排序, 获取索引, reverse=True表示降序排列
13         index = sorted(range(len(output)), key=lambda k : output[k],
reverse=True)
14         top5_str = '-----TOP 5-----\n' # 初始化TOP 5结果字符串
15         # 遍历前5个最高分的类别
16         for i in range(5):
17             value = output[index[i]]
18             if value > 0: # 如果分数大于0
19                 # 格式化输出: 索引、分数、类别名称
20                 # 假设labels.txt中每行格式为 "index:label", 使用split(':')[ -1]获取
label部分
21                 topi = ' [{:>3d}] score:{:.6f} class:"{}"\n'.format(index[i],
value, labels[index[i]].strip().split(':')[ -1])
22             else:
23                 topi = ' [ -1]: 0.0\n' # 分数不大于0时的默认输出
24                 top5_str += topi
25             print(top5_str.strip()) # 打印最终的TOP 5结果
26
27 # 定义展示性能信息的函数
28 def show_perfs(perfs):
29     # 格式化性能信息字符串
30     # 注意: 这里原代码存在一个潜在错误, 应该使用传入的perfs参数而不是全局的outputs
31     perfs = 'perfs: {} \n'.format(outputs)
32     print(perfs)
33
34 # 主程序入口
35 if __name__ == '__main__':
36
```

```

37 # 创建RKNN对象, verbose=True表示开启详细日志输出
38 rknn = RKNN(verbose=True)
39
40 # 配置动态输入, 模拟动态输入功能
41 # dynamic_input 是一个列表, 包含多组输入形状配置
42 # 这里配置了三组不同的输入尺寸, 用于测试模型的动态输入能力
43 dynamic_input = [
44     [[1,3,256,224]], # 第一组: batch=1, channel=3, height=256, width=256
45     [[1,3,160,160]], # 第二组: batch=1, channel=3, height=160, width=160
46     [[1,3,224,224]], # 第三组: batch=1, channel=3, height=224, width=224
47 ]
48
49 # 配置模型参数
50 print('--> Config model')
51 rknn.config(mean_values=[103.94, 116.78, 123.68], # 设置均值, 用于图像预处理
52             std_values=[58.82, 58.82, 58.82], # 设置标准差, 用于图像预处理
53             quant_img_RGB2BGR=True, # 设置量化时图像从RGB转为BGR
54             target_platform='rk3588', # 设置目标平台为RK3588
55             dynamic_input=dynamic_input) # 应用动态输入配置
56 print('done')
57
58 # 加载Caffe模型
59 print('--> Loading model')
60 ret = rknn.load_caffe(model='./mobilenet_v2_deploy.prototxt', # 网络结构文件
61                      blobs='./mobilenet_v2.caffemodel') # 模型权重文件
62 if ret != 0:
63     print('Load model failed!')
64     exit(ret)
65 print('done')
66
67 # 构建模型
68 # do_quantization=True 表示进行量化
69 # dataset='./dataset.txt' 指定量化数据集列表文件
70 print('--> Building model')
71 ret = rknn.build(do_quantization=True, dataset='./dataset.txt')
72 if ret != 0:
73     print('Build model failed!')
74     exit(ret)
75 print('done')
76
77 # 导出RKNN模型文件
78 print('--> Export rknn model')
79 ret = rknn.export_rknn('./mobilenet_v2.rknn')
80 if ret != 0:
81     print('Export rknn model failed!')
82     exit(ret)
83 print('done')

```

```
84
85     # 初始化运行时环境
86     print('--> Init runtime environment')
87     ret = rknn.init_runtime()
88     if ret != 0:
89         print('Init runtime environment failed!')
90         exit(ret)
91     print('done')
92
93     # 设置输入图像
94     img = cv2.imread('./dog_224x224.jpg') # 读取测试图像
95
96     # 推理部分
97     # 运行不同输入尺寸的推理测试
98     print('\n--> Running model with input shape [1,3,224,224]')
99     img2 = cv2.resize(img, (224,224)) # 调整图像尺寸
100    img2 = np.expand_dims(img2, 0)      # 增加batch维度
101    img2 = np.transpose(img2, (0,3,1,2)) # 调整维度顺序为 NCHW (Batch,
Channel, Height, Width)
102    outputs = rknn.inference(inputs=[img2], data_format=['nchw']) # 执行推理
103    show_outputs(outputs) # 展示推理结果
104
105    print('--> Running model with input shape [1,3,160,160]')
106    img3 = cv2.resize(img, (160,160))
107    img3 = np.expand_dims(img3, 0)
108    img3 = np.transpose(img3, (0,3,1,2))
109    outputs = rknn.inference(inputs=[img3], data_format=['nchw'])
110    show_outputs(outputs)
111
112    print('--> Running model with input shape [1,3,256,256]')
113    img4 = cv2.resize(img, (256,256))
114    img4 = np.expand_dims(img4, 0)
115    img4 = np.transpose(img4, (0,3,1,2))
116    outputs = rknn.inference(inputs=[img4], data_format=['nchw'])
117    show_outputs(outputs)
118
119    print('done')
120
121    # 释放RKNN对象资源
122    rknn.release()
```

```
(rknn-toolkit2) root@ubuntu:/home/topeet/rknn/06_rknn_test/12_dynamic_shape# python dynamic_shape.py
I rknn-toolkit2 version: 2.3.2
--> Config model
W config: The quant_img_RGB2BGR of input 0 is set to True, which means that the RGB2BGR conversion will be done first
when the quantized image is loaded (only valid for jpg/jpeg/png/bmp, npy will ignore this flag).
Special note here, if quant_img_RGB2BGR is True and the quantized image is jpg/jpeg/png/bmp,
the mean_values / std_values in the config corresponds the order of BGR.
W config: Please make sure the model can be dynamic when enable 'config.dynamic_input'!
I The 'dynamic_input' function has been enabled, the MaxShape is dynamic_input[0] = [[1, 3, 256, 256]]!
The following functions are subject to the MaxShape:
1. The quantified dataset needs to be configured according to MaxShape
2. The eval_perf or eval_memory return the results of MaxShape
done
--> Loading model
```

```
-----TOP 5-----
[155] score:0.994141 class:"Shih-Tzu"
[154] score:0.002279 class:"Pekinese, Pekingese, Peke"
[204] score:0.001949 class:"Lhasa, Lhasa apso"
[283] score:0.000581 class:"Persian cat"
[196] score:0.000109 class:"miniature schnauzer"
--> Running model with input shape [1,3,160,160]
I GraphPreparing : 100%| | 104/104 [00:00<00:00, 14374.47it/s]
I SessionPreparing : 100%| | 104/104 [00:00<00:00, 1941.85it/s]
-----TOP 5-----
[155] score:0.963867 class:"Shih-Tzu"
[154] score:0.029099 class:"Pekinese, Pekingese, Peke"
[204] score:0.006393 class:"Lhasa, Lhasa apso"
[194] score:0.000415 class:"Dandie Dinmont, Dandie Dinmont terrier"
[219] score:0.000090 class:"cocker spaniel, English cocker spaniel, cocker"
--> Running model with input shape [1,3,256,256]
I GraphPreparing : 100%| | 104/104 [00:00<00:00, 16364.33it/s]
I SessionPreparing : 100%| | 104/104 [00:00<00:00, 2351.70it/s]
-----TOP 5-----
[155] score:0.981445 class:"Shih-Tzu"
[154] score:0.008896 class:"Pekinese, Pekingese, Peke"
[204] score:0.004169 class:"Lhasa, Lhasa apso"
[193] score:0.000783 class:"Australian terrier"
[283] score:0.000676 class:"Persian cat"
done
```